

Chapter 29. Python as Software, Not Scripts

From a working script to a tested package: structure, lockfiles, types, tests, logging, robust API clients, and code review.

Draft chapter v1.1

Contents

Chapter 29. Python as Software, Not Scripts

Why this matters

In plain English

29.1 The script that works until it doesn't

29.2 Functions and modules: one job each

29.3 Project structure and configuration

29.4 Environments and lockfiles

29.5 Classes, when they help

29.6 Type hints

29.7 Testing with pytest

29.8 Errors and logging

29.9 Robust API clients

29.10 Code review

Common mistakes and how to spot them

In the real world: the report that said January was zero

Tools

The project: turn a script into a tested package

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 29. Python as Software, Not Scripts

Part III — Advanced Analytics & Analytics Engineering

Chapter at a glance

You will learn to: recognize the signs that a script has outgrown being a script · split a script into small functions and modules with one job each · lay out a project with `src/`, `tests/`, and `pyproject.toml` · keep passwords and settings out of code · create reproducible environments with `venv`, pinned requirements, and a lockfile managed by `uv` · use data classes where they help, and avoid classes where they don't · add type hints and let `mypy` find bugs before users do · write fast, focused tests with `pytest`, fixtures, and parametrized cases · raise meaningful exceptions and log instead of printing · build an API client that handles pagination, timeouts, retries, and rate limits · review code, and have your code reviewed, with a checklist.

Before you start: Chapter 17 (functions, files, errors, packages), Chapter 18 (pandas and the monthly report), Chapter 34 (the terminal, environment variables, exit codes), Chapter 12 (connecting to PostgreSQL), and Chapter 2 (what an API is). Chapter 26's Git basics help but aren't required.

Time needed: 12–16 hours of reading and practice, spread over two to three weeks.

Tools: Python 3.12 or later, `uv` (section 29.4), a code editor such as VS Code, and the `riverstone_2025` database from Chapter 13.

Practice data: the companion folder `ch29/`: the starting script `start/monthly_report.py`, the finished package `riverstone-report/`, and a local mock CRM API, `mock_crm_api.py`, with 43 leads. Every command was run, and every output shown is real.

Why this matters

The monthly report script from Chapter 18 does its job. It pulls December's sales, adds them up, and writes a workbook. Then this happens:

- "Can Priya run it while you're on leave?" It needs your password, your folder, and the exact versions of pandas on your laptop.
- "The January report says revenue was zero." The script found no data, wrote a report full of zeros anyway, printed `done`, and overwrote December's file.
- "Can we also pull leads from the CRM?" The CRM's API returns 10 leads per page, sometimes fails, and blocks you if you ask too often.
- "Finance changed the revenue rule. Did anything break?" Nobody knows until someone reads every number.

None of these are problems with Python. They're problems with treating code that a business depends on as a one-off script. This chapter turns the report into **software**: code that someone else can install, run, understand, change, and trust. The techniques are the everyday habits of data engineers and analytics engineers, and the questions interviewers use to tell them apart from people who have only written notebooks: "How do you structure a Python project?", "How would you test this function?", "What happens when the API returns a 429?"

In plain English

A script is a recipe scribbled on the back of an envelope. Software is a restaurant kitchen.

The envelope works when you cook alone. It says "a handful of salt" because you know your hand. It lists the steps in one long paragraph. If the shop is out of an ingredient, you improvise. Nobody else could cook from it.

A restaurant kitchen is built so that any trained cook can produce the same dish every night:

- The work is divided into **stations**: one prepares vegetables, one grills, one plates. Each has one job. In code, these are **functions** and **modules**.
 - Ingredients come from **named suppliers with fixed specifications**, so tonight's dish tastes like last week's. That's a **lockfile** pinning exact package versions.
 - The **head chef tastes every station's output** before it leaves the kitchen, the same way every time. Those are **tests**.
 - Recipes state **exactly what goes in and comes out** ("200 g, diced"), not "some onions". That's **type hints**.
 - When something goes wrong, the kitchen doesn't quietly serve a bad plate. It **stops and says why**, and someone writes it in the **log book**. Those are **exceptions** and **logging**.
 - Deliveries are late sometimes. The kitchen **calls again after a sensible wait**, not every ten seconds, and doesn't give up on the first try. That's a **robust API client**.
 - A new recipe isn't added to the menu until **another chef has read it**. That's **code review**.
-

29.1 The script that works until it doesn't

Here is the starting point, `start/monthly_report.py` in the companion files. It's a typical first version of Chapter 18's monthly report, and it runs:

```

# monthly_report.py - Riverstone's monthly sales report, as a first working script.
# Analyst to Architect · Chapter 29 · the "before" version: it works, and it has every problem
section 29.1 lists.
# How: python3 monthly_report.py    (needs riverstone_2025 in PostgreSQL, pandas, openpyxl,
SQLAlchemy, psycopg)
# Tested on: Python 3.12.3, pandas 3.0.2, openpyxl 3.1.5, SQLAlchemy 2.0, psycopg 3.3.5, PostgreSQL
16.
# Riverstone Supplies is fictional; every name and number is invented.
import pandas as pd
from sqlalchemy import create_engine

conn = create_engine("postgresql+psycopg://postgres:riverstone123@localhost/riverstone_2025")
month = "2025-12"

df = pd.read_sql("SELECT sl.*, c.customer_name, p.category AS cat FROM sales_lines sl "
                "JOIN customers c ON c.customer_id = sl.customer_id "
                "JOIN products p ON p.product_id = sl.product_id", conn)
df["order_date"] = pd.to_datetime(df["order_date"])
df = df[df["order_date"].dt.strftime("%Y-%m") == month]

total = df["net_revenue"].astype(float).sum()
orders = df["order_id"].nunique()
print("Revenue:", total)
print("Orders:", orders)

try:
    t = pd.read_sql("SELECT target_revenue FROM sales_targets WHERE target_month = '" + month +
"-01'", conn)
    target = float(t.iloc[0, 0])
except:
    target = 0

cats = df.groupby("cat")["net_revenue"].sum().astype(float).reset_index()
cats["share"] = cats["net_revenue"] / total
cats = cats.sort_values("net_revenue", ascending=False)

top = df.groupby("customer_name")
["net_revenue"].sum().astype(float).sort_values(ascending=False).head(5).reset_index()

with pd.ExcelWriter("Monthly_Report_Dec_FINAL.xlsx") as w:
    pd.DataFrame({"metric": ["Revenue", "Orders", "Target", "% of target"],
                  "value": [total, orders, target, total / target * 100]}).to_excel(w,
sheet_name="Summary", index=False)
    cats.to_excel(w, sheet_name="Categories", index=False)
    top.to_excel(w, sheet_name="Top customers", index=False)
print("done")

```

```

# terminal, in companion/ch29/start
$ python3 monthly_report.py
Revenue: 439823.5
Orders: 18
done

```

December's numbers are right: ₹439,823.50 from 18 orders, as Chapter 13's queries found. Now it's the first week of February, and someone changes `month` to `"2026-01"` to produce January's report. Riverstone's `riverstone_2025` database ends on 31 December 2025, so there is no January data, exactly as there would be if the nightly load had failed:

```
$ python3 monthly_report.py
Revenue: 0.0
Orders: 0
/home/meera/ch29/start/monthly_report.py:37: RuntimeWarning: invalid value encountered in scalar
divide
  "value": [total, orders, target, total / target * 100])).to_excel(w, sheet_name="Summary",
index=False)
done
```

It printed `done`. The exit code was 0, so a scheduler would record success. And because the file name is fixed, it replaced `Monthly_Report_Dec_FINAL.xlsx`, the good December report, with a January report showing zero revenue and a percentage of target that isn't a number.

Here's every problem in those 40 lines, and the section that fixes it:

Problem in the script	Why it hurts	Fixed in
Everything runs at the top level, in one file	Can't test a calculation without a database, can't reuse a piece	29.2, 29.3
Password and connection details in the code	Anyone with the file has the password; it ends up in Git	29.3
Month and file name typed into the code	Every run means editing code; files overwrite each other	29.3, 29.5
No record of which package versions it needs	"Works on my machine"; breaks after an upgrade (it already broke once: pandas 3 requires <code>sheet_name=</code> by name)	29.4
No tests	Nobody knows whether a change broke a number	29.7
<code>except:</code> catches everything and sets <code>target = 0</code>	Hides the real cause, and produces a wrong percentage	29.8
<code>print</code> instead of logging; exit code 0 on failure	Scheduled runs look successful when they aren't	29.8
SQL built by gluing strings (<code>"... = '" + month + "-01'"</code>)	Breaks on unexpected input, and is how SQL injection happens	29.2
Loads every order line of the year, then filters in pandas	Fine for 326 lines, slow for 2 million (Chapter 28)	29.2

Simplification note. Scripts aren't bad. A script you run once, or a notebook for exploring data, doesn't need a package, tests, and a lockfile. The signal to change is when **other people depend on the output, it runs on a schedule, or it will be changed by someone other than its author**. Section 29.10 comes back to this judgment.

29.2 Functions and modules: one job each

The first step is always the same: pull each job into a **function** with a clear name, inputs, and a return value. A function that only calculates from its inputs, without reading files, databases, clocks, or global variables, and without changing anything outside itself, is called a **pure function**. Pure functions are

the easiest code in the world to test, because the same input always gives the same output.

The report has four jobs, and they separate cleanly:

1. **Configure**: which month, which database, where to write.
2. **Read**: fetch the month's sales lines and target from the database.
3. **Transform**: summarize, break down by category, rank customers. *Pure*.
4. **Write**: produce the formatted workbook.

Each job goes in its own **module**, a `.py` file you can `import`. Here is the transform module, the heart of the report:

```

"""Pure calculations: DataFrames in, results out. No database, no files, so they are easy to
test."""
from __future__ import annotations

from dataclasses import dataclass

import pandas as pd

from riverstone_report.errors import NoDataError

REQUIRED_COLUMNS = {"order_id", "customer_name", "category", "net_revenue"}

@dataclass(frozen=True)
class MonthSummary:
    revenue: float
    orders: int
    customers: int
    target: float | None

    @property
    def pct_of_target(self) -> float | None:
        if not self.target:
            return None
        return round(100 * self.revenue / self.target, 1)

def check_lines(lines: pd.DataFrame) -> None:
    missing = REQUIRED_COLUMNS - set(lines.columns)
    if missing:
        raise ValueError(f"sales lines are missing columns: {sorted(missing)}")
    if lines.empty:
        raise NoDataError("no sales lines for this month")

def summarize(lines: pd.DataFrame, target: float | None) -> MonthSummary:
    check_lines(lines)
    return MonthSummary(
        revenue=round(float(lines["net_revenue"].sum()), 2),
        orders=int(lines["order_id"].nunique()),
        customers=int(lines["customer_name"].nunique()),
        target=target,
    )

def revenue_by_category(lines: pd.DataFrame) -> pd.DataFrame:
    check_lines(lines)
    result = (lines.groupby("category", as_index=False)
              .agg(net_revenue=("net_revenue", "sum"))
              .sort_values(["net_revenue", "category"], ascending=[False, True],
                           ignore_index=True))
    result["share_pct"] = (100 * result["net_revenue"] / result["net_revenue"].sum()).round(1)
    return result

def top_customers(lines: pd.DataFrame, n: int = 5) -> pd.DataFrame:
    if n < 1:
        raise ValueError("n must be at least 1")
    check_lines(lines)
    totals = lines.groupby("customer_name", as_index=False).agg(net_revenue=("net_revenue", "sum"))
    ranked = totals.sort_values(["net_revenue", "customer_name"], ascending=[False, True],
                                ignore_index=True)
    return ranked.head(n)

```

Because these functions only need a DataFrame, you can try them on five made-up lines in a notebook or the Python prompt, without a database:

```
import sys
sys.path[:0] = ["riverstone-report/src", "."] # lets Python find the package and mock_crm_api
from the companion folder

import pandas as pd
from riverstone_report.transform import summarize, revenue_by_category, top_customers

lines = pd.DataFrame({
    "order_id":      [1, 1, 2, 3, 3],
    "customer_name": ["Sharma Hardware", "Sharma Hardware", "Metro Mart", "Green Leaf Hotels",
"Green Leaf Hotels"],
    "category":      ["Storage", "Kitchen", "Storage", "Kitchen", "Furniture"],
    "net_revenue":   [10000.0, 2500.0, 6000.0, 1500.0, 5000.0],
})

summary = summarize(lines, target=20000.0)
print(summary)
print(summary.pct_of_target)
print(revenue_by_category(lines))
```

```
MonthSummary(revenue=25000.0, orders=3, customers=3, target=20000.0)
125.0
   category  net_revenue  share_pct
0   Storage      16000.0         64.0
1  Furniture       5000.0         20.0
2   Kitchen       4000.0         16.0
```

How it works:

- `summarize` counts **orders** with `nunique()` , not lines: five lines, three orders. That's Chapter 12's grain lesson, now written once in a function everyone uses.
- `pct_of_target` is a **property**: it's calculated when you ask for it, from the other fields, so it can never disagree with them. $25,000 \div 20,000 = 125.0\%$. ✓
- The category shares add to 100.0% (64.0 + 20.0 + 16.0). ✓ Sorting by revenue *and then* by name makes the order stable when two categories tie, the same reason Chapter 13 added tie-breakers to `ROW_NUMBER` .
- `check_lines` raises an exception instead of returning zeros for an empty month. Section 29.8 explains why that's the right behavior.

The `read` module is the only place that knows SQL. It fixes two problems from the script: it asks the database for **only the month's rows**, and it passes the dates as **query parameters** (`:month_start`), so values are never glued into the SQL text:

```

"""Reading from the database. The only module that knows SQL."""
from __future__ import annotations

from datetime import date

import pandas as pd
from sqlalchemy import Engine, text

SALES_LINES_SQL = text("""
    SELECT sl.order_id, sl.order_date, c.customer_name, sl.category, sl.net_revenue
    FROM sales_lines AS sl
    JOIN customers AS c ON c.customer_id = sl.customer_id
    WHERE sl.order_date >= :month_start AND sl.order_date < :next_month
""")

TARGET_SQL = text("SELECT target_revenue FROM sales_targets WHERE target_month = :month_start")

def fetch_sales_lines(engine: Engine, month_start: date, next_month: date) -> pd.DataFrame:
    """One row per non-cancelled order line in the month, with net_revenue as a float."""
    with engine.connect() as conn:
        lines = pd.read_sql(SALES_LINES_SQL, conn, params={"month_start": month_start,
"next_month": next_month})
        return lines.astype({"net_revenue": "float64"})

def fetch_target(engine: Engine, month_start: date) -> float | None:
    """The month's revenue target, or None if no target was set."""
    with engine.connect() as conn:
        value = conn.execute(TARGET_SQL, {"month_start": month_start}).scalar_one_or_none()
        return None if value is None else float(value)

```

Watch out: never build SQL by adding strings. `"WHERE target_month = '" + month + "-01'"` works until `month` contains a quote, or until someone passes `2025-12' OR '1'='1`. That's **SQL injection**, one of the most common security holes in software. Parameters (`:month_start` with SQLAlchemy's `text`, `%s` with `psycopg` directly) send the value separately from the SQL, so it's always treated as a value.

```
if __name__ == "__main__"
```

When Python runs a file directly, it sets the special variable `__name__` to `"__main__"`. When the same file is imported, `__name__` is the module's name. So code under `if __name__ == "__main__":` runs only when the file is executed, not when a test imports it. The script had all its code at the top level, so importing it to test one calculation would have connected to the database and written a file. Every module in the package defines functions and nothing else at the top level; only `cli.py` has a `__main__` block.

29.3 Project structure and configuration

The layout

```
# terminal, in companion/ch29/riverstone-report (hidden tool folders omitted)
$ find . -type f | sort
./gitignore
./python-version
./README.md
./pyproject.toml
./src/riverstone_report/__init__.py
./src/riverstone_report/cli.py
./src/riverstone_report/config.py
./src/riverstone_report/crm_client.py
./src/riverstone_report/db.py
./src/riverstone_report/errors.py
./src/riverstone_report/excel.py
./src/riverstone_report/headline.py
./src/riverstone_report/transform.py
./tests/conftest.py
./tests/test_config.py
./tests/test_crm_client.py
./tests/test_reconcile.py
./tests/test_transform.py
./uv.lock
```

From one script to a package

Before: `monthly_report.py` (40 lines)

connect with a password in the code

hard-coded month and file name

load every order line, filter in pandas

calculate, print

bare except: target = 0

write `Monthly_Report_Dec_FINAL.xlsx`

print('done')

Everything in one file, run top to bottom.



After: `riverstone-report/`

<code>pyproject.toml</code>	dependencies, tools, entry point
<code>uv.lock</code>	exact versions of 32 packages
<code>src/riverstone_report/</code>	
<code>config.py</code>	settings from env vars, validated
<code>db.py</code>	the only module with SQL
<code>transform.py</code>	pure calculations
<code>excel.py</code>	formatted workbook
<code>crm_client.py</code>	retries, pages, rate limits
<code>cli.py</code>	<code>riverstone-report --month</code>
<code>errors.py</code>	<code>ReportError</code> and friends
<code>tests/</code>	23 tests

Each file has one job, so each can be read, tested, and changed alone.

Figure 29.1 – The same report, split so that each file has one job.

- A **package** is a folder of modules that Python can import as one unit. `src/riverstone_report/` is the package; `__init__.py` marks it and says what it offers.
- The **src/** **layout** puts the package one level down, so tests import the *installed* package rather than whatever happens to be in the current folder. It catches a whole class of "passes on my machine" mistakes, and it's what `uv init --package` creates.
- **tests/** sits beside `src/`, not inside it, so tests aren't shipped with the package.
- **pyproject.toml** is the standard file describing a Python project: its name, version, dependencies, entry points, and settings for tools like `pytest` and `mypy`.
- **.gitignore** keeps the virtual environment, caches, and generated reports out of version control.

Here's the project's `pyproject.toml`, created by `uv init --package riverstone-report` and filled in by `uv add` (section 29.4):

```
[project]
name = "riverstone-report"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "openpyxl>=3.1.5",
    "pandas>=3.0.5",
    "psycpg[binary]>=3.3.5",
    "requests>=2.34.2",
    "sqlalchemy>=2.0.54",
]

[project.scripts]
riverstone-report = "riverstone_report:main"

[build-system]
requires = ["uv_build>=0.12.15,<0.13.0"]
build-backend = "uv_build"

[dependency-groups]
dev = [
    "mypy>=2.3.1",
    "pandas-stubs>=3.0.5.260914",
    "pytest>=9.1.1",
    "types-openpyxl>=3.1.5.20260827",
    "types-requests>=2.33.0.20260906",
]

[tool.pytest.ini_options]
testpaths = ["tests"]

[tool.mypy]
strict = true
files = ["src"]
```

The `[project.scripts]` entry creates a command called `riverstone-report` when the package is installed. It runs the `main` function that `__init__.py` imports from `cli.py`.

Configuration: settings belong outside the code

The script had three kinds of setting mixed into its code: the **secret** (the password), the **environment** (which database), and the **run** (which month). They change for different reasons and at different times, so they come from different places:

Setting	Changes when	Comes from
Month to report	every run	a command-line argument: <code>--month 2025-12</code>
Database URL (with password)	per machine or server; must stay secret	an environment variable , <code>RIVERSTONE_DATABASE_URL</code>
Output folder	per machine	an environment variable with a default, <code>RIVERSTONE_OUTPUT_DIR</code>
Number of top customers	rarely; a business decision	a default in code

An **environment variable** is a named value that the operating system passes to every program you start from that terminal, as Chapter 34 showed. Setting one:

```
# macOS or Linux terminal
export
RIVERSTONE_DATABASE_URL="postgresql+psycopg://postgres:YOUR_PASSWORD@localhost/riverstone_2025"

# Windows PowerShell
$env:RIVERSTONE_DATABASE_URL =
"postgresql+psycopg://postgres:YOUR_PASSWORD@localhost/riverstone_2025"
```

Many teams keep these in a **.env file** that's listed in `.gitignore` and loaded at start-up (the `python-dotenv` package does this). On servers and schedulers, the platform's **secrets manager** supplies them instead. Either way, the rule is the same: **a password never appears in a file that goes into version control.**

Watch out: a password committed once is exposed forever. Deleting it in a later commit doesn't remove it from Git's history, and public repositories are scanned for credentials within minutes. If a secret is ever committed, treat it as leaked: change the password first, then clean up.

29.4 Environments and lockfiles

Why "pip install pandas" isn't enough

Every Python project depends on packages, and those packages depend on others. The report needs 5 packages directly, and they pull in 11 more. When you type `pip install pandas`, you get whatever version is newest *today*, installed into whatever Python you happen to be using, shared with every other project on the machine. Six months later a colleague installs the "same" requirements and gets pandas 3.1 instead of 3.0, and something behaves differently. The starting script has already been bitten once: written for pandas 2, it stopped running under pandas 3, because `to_excel` now requires the sheet name to be passed by keyword.

Two tools solve two different problems:

- A **virtual environment** is a private folder of packages for one project, with its own copy of the `python` and `pip` commands. Projects stop interfering with each other.
- A **lockfile** records the **exact version of every package**, including the indirect ones, so every installation is identical.

The classic way: venv and pinned requirements

Python ships with `venv`. It works everywhere, including locked-down company laptops, so learn it first:

```
# macOS or Linux terminal, in your project folder
python3 -m venv .venv          # create the environment in a folder called .venv
source .venv/bin/activate      # use it (Windows PowerShell: .venv\Scripts\Activate.ps1)
pip install pandas openpyxl sqlalchemy "psycopg[binary]" requests
pip freeze > requirements.txt  # record the exact versions installed
deactivate                    # stop using it
```

A colleague then runs `python3 -m venv .venv`, activates it, and `pip install -r requirements.txt`. `pip freeze` records exact versions (`pandas==3.0.5`) of everything installed, which makes it a simple lockfile. Its weaknesses: it can't tell which packages you asked for and which came along as dependencies, it doesn't record which versions work on other operating systems, and keeping it up to date is manual.

The modern way: uv, pyproject.toml, and uv.lock

uv is a fast, free package and project manager from Astral that does the whole job: it creates the virtual environment, resolves versions, writes a cross-platform lockfile, and runs commands inside the environment. It was version 0.12.15 when this chapter was tested. Install it from the official instructions at `docs.astral.sh/uv` (a one-line installer on each operating system, or `pip install uv`).

The whole setup of this chapter's package was four commands:

```
# terminal, in companion/ch29
uv init --package riverstone-report
cd riverstone-report
uv add pandas openpyxl sqlalchemy "psycopg[binary]" requests
uv add --dev pytest mypy pandas-stubs types-requests types-openpyxl
```

- `uv init --package` creates the `src/` layout, `pyproject.toml`, `.gitignore`, and `.python-version`.
- `uv add` adds a package to `pyproject.toml`'s dependencies (with a minimum version, `pandas>=3.0.5`), resolves every indirect dependency, writes **`uv.lock`**, and installs everything into `.venv`.
- `uv add --dev` puts tools that are only needed while developing (tests, type checking) in a separate **dependency group**, so they aren't installed when the report runs on a server.

`pyproject.toml` says what the project *needs*; `uv.lock` says exactly what was *installed*. Here's one entry from the lockfile:

```
name = "pandas"
version = "3.0.5"
source = { registry = "https://pypi.org/simple" }
dependencies = [
```

Both files go into version control. The payoff is on someone else's machine. Here's a fresh copy of the project (no `.venv`) being set up from the lockfile:

```
# terminal, in a fresh copy of the project
$ uv sync --locked
Using CPython 3.12.3 interpreter at: /usr/bin/python3.12
Creating virtual environment at: .venv
Resolved 32 packages in 1ms
  Building riverstone-report @ file:///home/meera/riverstone-report
  Built riverstone-report @ file:///home/meera/riverstone-report
Prepared 1 package in 5ms
Installed 30 packages in 461ms
+ ast-serialize==0.11.2
+ certifi==2026.7.22
+ charset-normalizer==3.5.1
+ et-xmlfile==2.0.0
+ greenlet==3.5.6
...
$ uv run python -c "import pandas, openpyxl, requests; print(pandas.__version__,
openpyxl.__version__, requests.__version__)"
3.0.5 3.1.5 2.34.2
```

(The list of installed packages is shortened here; the full output lists all 30, including pandas 3.0.5 and requests 2.34.2.)

`--locked` means "install exactly what the lockfile says, and fail if `pyproject.toml` and `uv.lock` disagree". That's the command for servers and automated test runs. `uv run` runs any command inside the project's environment without activating it: `uv run pytest`, `uv run mypy`, `uv run riverstone-report --month 2025-12`.

If a system needs a `requirements.txt` (some older deployment tools do), `uv export --no-dev --no-hashes` produces one from the lockfile.

Task	venv + pip	uv
Create environment	<code>python3 -m venv .venv</code>	automatic on <code>uv sync</code> , <code>uv add</code> , or <code>uv run</code>
Add a package	<code>pip install pandas</code> , then update <code>requirements.txt</code> by hand	<code>uv add pandas</code>
Record exact versions	<code>pip freeze > requirements.txt</code>	<code>uv.lock</code> , updated automatically
Install on another machine	<code>pip install -r requirements.txt</code>	<code>uv sync --locked</code>
Run a tool	activate, then <code>pytest</code>	<code>uv run pytest</code>
Upgrade one package	<code>pip install -U pandas</code> , <code>refreeze</code>	<code>uv lock --upgrade-package pandas</code> , then <code>uv sync</code>

Tool note. Poetry and PDM are older project managers that do similar jobs with their own lockfiles, and you'll meet them in existing company projects. conda manages non-Python dependencies too and is common in scientific computing. The ideas transfer: a project file for what you need, a lockfile for what you got, an isolated environment, and one command to reproduce it.

29.5 Classes, when they help

A **class** bundles data with the functions that work on it. Analysts coming from notebooks either never write one, or suddenly write classes for everything. The useful middle is small.

Data classes: named bundles of values

The report passes the same four settings everywhere. As loose variables, a function call looks like `run("2025-12", url, "reports", 5)`, and nothing stops the arguments arriving in the wrong order. A **data class** gives the bundle a name and typed fields, and writes the boilerplate (`__init__`, a readable `repr`, equality) for you:

```
"""Settings for one report run, read from arguments and environment variables."""
from __future__ import annotations

import os
from collections.abc import Mapping
from dataclasses import dataclass
from datetime import date
from pathlib import Path

from riverstone_report.errors import ConfigError

def parse_month(text: str) -> date:
    """Turn '2025-12' into date(2025, 12, 1). Raise ConfigError for anything else."""
    try:
        year, month = (int(part) for part in text.split("-"))
        return date(year, month, 1)
    except ValueError:
        raise ConfigError(f"month must look like YYYY-MM, got {text!r}") from None

@dataclass(frozen=True)
class ReportConfig:
    month: date
    database_url: str
    output_dir: Path = Path("reports")
    top_n: int = 5

    @classmethod
    def from_env(cls, month: str, env: Mapping[str, str] = os.environ) -> ReportConfig:
        url = env.get("RIVERSTONE_DATABASE_URL")
        if not url:
            raise ConfigError("set RIVERSTONE_DATABASE_URL to the database connection URL")
        return cls(
            month=parse_month(month),
            database_url=url,
            output_dir=Path(env.get("RIVERSTONE_OUTPUT_DIR", "reports")),
        )

    @property
    def next_month(self) -> date:
        if self.month.month == 12:
            return date(self.month.year + 1, 1, 1)
        return date(self.month.year, self.month.month + 1, 1)

    @property
    def output_path(self) -> Path:
        return self.output_dir / f"riverstone_monthly_{self.month:%Y-%m}.xlsx"
```

Try it:

```
from riverstone_report.config import ReportConfig, parse_month
from riverstone_report.errors import ConfigError

config = ReportConfig.from_env("2025-12", env={"RIVERSTONE_DATABASE_URL":
"postgresql+psycopg://user:secret@db.example/riverstone"})
print(config.month, config.next_month, config.output_path)

try:
    parse_month("Dec 2025")
except ConfigError as exc:
    print("ConfigError:", exc)

config.month = parse_month("2026-01")
```

```
2025-12-01 2026-01-01 reports/riverstone_monthly_2025-12.xlsx
ConfigError: month must look like YYYY-MM, got 'Dec 2025'
dataclasses.FrozenInstanceError: cannot assign to field 'month'
```

How it works:

- `@dataclass(frozen=True)` makes instances **immutable**: once a run's configuration is built, nothing can change it halfway through, so the last line raises `FrozenInstanceError`.
- `@classmethod from_env` is an **alternative constructor**: a named way to build the object from somewhere specific. The `env` parameter defaults to the real `os.environ`, but a test can pass a plain dictionary, as this example does. No test has to change your real environment variables.
- `next_month` and `output_path` are **properties**, so December correctly rolls into January of the next year, and the file name always contains the month. Two runs for different months can't overwrite each other.
- `parse_month` turns any bad input into one kind of error with a clear message. `from None` hides the less helpful internal `ValueError`.

A class with behavior: the API client

Section 29.9's `CrmClient` is a class for a different reason: it holds **state that several methods share** (the base URL, the API key, an open network session, retry settings) and it's created once and used many times. That's the test: **use a class when data and the functions that use it travel together**. For a calculation, a function is simpler. `transform.py` has no classes except the `MonthSummary` data class, and doesn't need any.

Watch out: don't store what you can calculate. If `MonthSummary` stored `pct_of_target` as a field, someone could change `revenue` and forget to update it. As a property, it's calculated from the current values every time. This is Chapter 28's normalization rule (store each fact once) applied to code.

29.6 Type hints

A **type hint** declares what a variable, parameter, or return value is supposed to be: `def parse_month(text: str) -> date:`. Python **doesn't enforce** hints when the code runs; they're documentation that tools can check. Their value comes from a **type checker**, a program that reads your code without running it and reports places where the types can't work.

The common hints you'll use:

Hint	Means
<code>int, float, str, bool, date</code>	that type
<code>list[str], dict[str, int], tuple[float, float]</code>	a container of that type
<code>float None</code>	a float, or <code>None</code> (older code writes <code>Optional[float]</code>)
<code>pd.DataFrame</code>	a pandas DataFrame (checked with the <code>pandas-stubs</code> package)
<code>Callable[[float], None]</code>	a function taking a float and returning nothing
<code>-> None</code>	the function returns nothing

Here's the bug a type checker is best at catching. Someone adds a function to write a one-line headline for emails:

```
def headline(summary: MonthSummary) -> str:
    gap = summary.pct_of_target - 100
    return f"Revenue ₹{summary.revenue:,.0f}, {gap:+.1f} points against target"
```

It works for December. Run `mypy`, the most widely used Python type checker, in strict mode (configured in `pyproject.toml`):

```
# terminal, in companion/ch29/riverstone-report
$ uv run mypy
src/riverstone_report/headline.py:6: error: Unsupported operand types for - ("None" and "int")
[operator]
src/riverstone_report/headline.py:6: note: Left operand is of type "float | None"
Found 1 error in 1 file (checked 9 source files)
```

`pct_of_target` is `float | None`, because a month can have no target. For such a month the function would crash with a `TypeError`, on the day the email goes out. `mypy` found it without a database, a test, or a run. The fix handles the `None` case explicitly:

```
"""A one-line headline for emails (section 29.6)."""
from riverstone_report.transform import MonthSummary

def headline(summary: MonthSummary) -> str:
    if summary.pct_of_target is None:
        return f"Revenue ₹{summary.revenue:,.0f} (no target set)"
    gap = summary.pct_of_target - 100
    return f"Revenue ₹{summary.revenue:,.0f}, {gap:+.1f} points against target"
```

```
$ uv run mypy
Success: no issues found in 9 source files
```

Type checking earned its place in the package: making the transforms pass strict mode also forced `groupby` results to be built with `.agg(...)`, which the pandas type stubs know return DataFrames.

Tool note. Your editor uses the same hints: VS Code's Python extension underlines type errors as you type, and completes `summary.` with the right fields. Other checkers include **Pyright** (used by VS Code) and newer, faster ones appearing each year. Hints cost a few characters per function and pay back on every change, so add them to any code that more than one person maintains.

29.7 Testing with pytest

A **test** is code that runs your code with known inputs and checks the result. **pytest** is the standard Python test runner: it finds files named `test_*.py`, runs every function named `test_*`, and reports what failed. Tests turn "I think it still works" into a command anyone can run.

A first test

```
import pandas as pd
import pytest

from riverstone_report.errors import NoDataError
from riverstone_report.transform import MonthSummary, revenue_by_category, summarize, top_customers

def test_summarize_counts_orders_not_lines(lines):
    summary = summarize(lines, target=20000.0)
    assert summary.revenue == 25000.0
    assert summary.orders == 3
    assert summary.customers == 3
    assert summary.pct_of_target == 125.0
```

A test is an ordinary function with `assert` statements. When an `assert` is false, pytest shows exactly which values disagreed. The five-line DataFrame is small enough that you can check every expected number by hand: Sharma Hardware's two lines are one order (₹12,500), so three orders in total, and ₹25,000 of ₹20,000 is 125.0%.

Fixtures: shared test data

The `lines` parameter isn't defined in the test file. It's a **fixture**: a function in `tests/conftest.py`, marked `@pytest.fixture`, that builds the test data. Pytest sees a test asking for `lines` and passes in a fresh copy for each test, so one test can't spoil the data for the next:

```

import pandas as pd
import pytest

@pytest.fixture
def lines() -> pd.DataFrame:
    """Five order lines from three orders: small enough to check every number by hand."""
    return pd.DataFrame({
        "order_id":      [1, 1, 2, 3, 3],
        "customer_name": ["Sharma Hardware", "Sharma Hardware", "Metro Mart", "Green Leaf Hotels",
"Green Leaf Hotels"],
        "category":      ["Storage", "Kitchen", "Storage", "Kitchen", "Furniture"],
        "net_revenue":    [10000.0, 2500.0, 6000.0, 1500.0, 5000.0],
    })

```

Testing many cases and testing failures

```

from datetime import date
from pathlib import Path

import pytest

from riverstone_report.config import ReportConfig, parse_month
from riverstone_report.errors import ConfigError

@pytest.mark.parametrize("text, expected", [("2025-12", date(2025, 12, 1)), ("2026-01", date(2026,
1, 1))])
def test_parse_month(text, expected):
    assert parse_month(text) == expected

@pytest.mark.parametrize("bad", ["Dec 2025", "2025-13", "2025", ""])
def test_parse_month_rejects_bad_input(bad):
    with pytest.raises(ConfigError):
        parse_month(bad)

def test_from_env_needs_database_url():
    with pytest.raises(ConfigError, match="RIVERSTONE_DATABASE_URL"):
        ReportConfig.from_env("2025-12", env={})

def test_december_rolls_into_next_year():
    config = ReportConfig.from_env("2025-12", env={"RIVERSTONE_DATABASE_URL":
"postgresql://example"})
    assert config.next_month == date(2026, 1, 1)
    assert config.output_path == Path("reports/riverstone_monthly_2025-12.xlsx")

```

- **@pytest.mark.parametrize** runs one test function once per set of values. Four bad month formats are four separate test results, and if one fails, pytest names the input that failed.
- **pytest.raises** checks that code fails *the right way*. Testing that `parse_month("2025-13")` raises `ConfigError` is as important as testing that `"2025-12"` works, because the original script's bugs were all on the failure path.
- `env={}` tests a missing environment variable without touching the real environment.

Running the tests

```
# terminal, in the project folder, with RIVERSTONE_DATABASE_URL set
$ uv run pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/meera/riverstone-report
configfile: pyproject.toml
testpaths: tests
collected 23 items

tests/test_config.py ..... [ 34%]
tests/test_crm_client.py ..... [ 56%]
tests/test_reconcile.py . [ 60%]
tests/test_transform.py ..... [100%]

===== 23 passed in 1.58s =====
```

Each dot is a passing test. 23 tests in under two seconds, so there's no reason not to run them after every change.

What a failure looks like

Suppose someone "simplifies" `summarize` and writes `orders=len(lines)`, counting lines instead of orders:

```
$ uv run pytest -q --tb=short tests/test_transform.py
F..... [100%]
===== FAILURES =====
_____ test_summarize_counts_orders_not_lines _____
tests/test_transform.py:11: in test_summarize_counts_orders_not_lines
    assert summary.orders == 3
E   assert 5 == 3
E   + where 5 = MonthSummary(revenue=25000.0, orders=5, customers=3, target=20000.0).orders
===== short test summary info =====
FAILED tests/test_transform.py::test_summarize_counts_orders_not_lines - asse...
1 failed, 8 passed in 0.26s
```

The test's name says what rule was broken, and the output shows the wrong value (5) and where it came from. This is the fan-out mistake from Chapter 12, caught in a quarter of a second instead of in a board meeting.

The integration test: reconcile to a known number

Unit tests use made-up data. One test should prove the whole path works against the real database, and the best check is one this book already trusts: December 2025's net revenue of ₹439,823.50, 115.7% of its ₹380,000 target (Chapter 13).

```

"""Integration test: runs only when RIVERSTONE_DATABASE_URL points at riverstone_2025."""
import os

import pytest
from sqlalchemy import create_engine

from riverstone_report import db, transform
from riverstone_report.config import ReportConfig

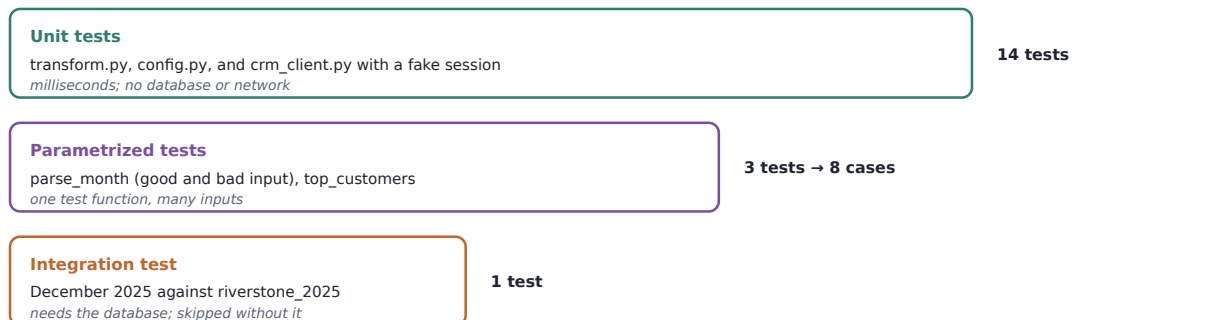
pytestmark = pytest.mark.skipif(not os.environ.get("RIVERSTONE_DATABASE_URL"), reason="no database
configured")

def test_december_2025_matches_the_book():
    config = ReportConfig.from_env("2025-12")
    engine = create_engine(config.database_url)
    lines = db.fetch_sales_lines(engine, config.month, config.next_month)
    summary = transform.summarize(lines, db.fetch_target(engine, config.month))
    assert summary.revenue == 439823.50
    assert summary.pct_of_target == 115.7
    assert transform.revenue_by_category(lines)["net_revenue"].sum() ==
pytest.approx(summary.revenue)

```

`pytestmark = pytest.mark.skipif(...)` skips the whole file when no database is configured, so the fast unit tests still run anywhere (a colleague's laptop, an automated check on every change) and show `s` for skipped instead of failing.

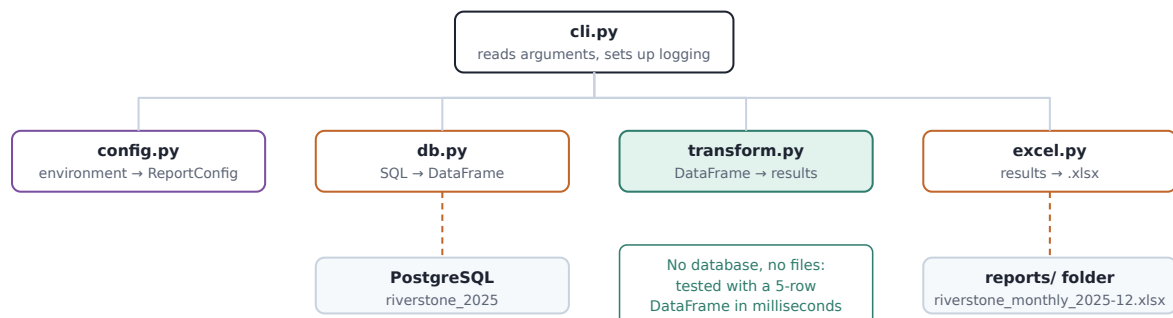
What each kind of test is for



The integration test reconciles to a number the book already trusts: December 2025 net revenue ₹439,823.50, 115.7% of target.

Figure 29.3 – Many fast tests on the pure core, a few slow ones on the edges.

Keep the calculations pure: input and output live at the edges



Orange modules touch the outside world and get a few integration tests. The green core holds the business rules and gets many fast unit tests.

Figure 29.2 – The design that makes testing cheap: business rules in pure functions, input and output at the edges.

What to test

- **Business rules:** every calculation someone would argue about (what counts as an order, how shares round, what happens with no target).
- **Edge cases:** empty input, a single row, ties, the last month of the year, `None`.
- **Failures:** bad input raises the right exception with a helpful message.
- **One reconciliation** against a known total, run when the database is available.
- **Not** third-party libraries (pandas' `groupby` is tested by pandas), and not every line for its own sake. A number called "coverage" measures which lines tests ran (`pytest-cov` reports it); treat it as a way to find untested code, not as a target.

Python link. Chapter 47 applies the same idea to data itself: automated checks that run against every day's data, not only against your code.

29.8 Errors and logging

Fail loudly, with a reason

The script's worst line was `except: target = 0`. It caught *every* possible error (a typo in the SQL, a lost connection, even the user pressing Ctrl+C), replaced it with a plausible-looking wrong value, and carried on. The rules the package follows instead:

1. **Catch only exceptions you can do something about**, by name: `except ConfigError:`, never a bare `except:`.
2. **Don't turn an error into a default value** unless the default is truly correct. A missing target isn't zero; it's `None`, and the report says "no target set".
3. **Define your own exceptions** for failures the business cares about, so callers can handle them as a group.
4. **Stop at the top:** one place (the command-line entry point) catches `ReportError`, logs it, and exits with a non-zero **exit code**, the number a program returns to say whether it succeeded (0) or failed (anything else). Schedulers, CI systems, and shell scripts read it.

```

"""Exceptions the report can raise. Callers catch ReportError to handle any of them."""

class ReportError(Exception):
    """Base class for every expected failure of the monthly report."""

class ConfigError(ReportError):
    """A setting is missing or invalid."""

class NoDataError(ReportError):
    """The requested month has no sales, so a report would be misleading."""

class CrmApiError(ReportError):
    """The CRM API kept failing after all retries, or refused the request."""

```

Because `NoDataError` and `ConfigError` both inherit from `ReportError`, except `ReportError` handles all of them, while a genuine bug (a `KeyError`, say) is not caught and crashes with a full traceback, which is what you want for a bug.

```

from riverstone_report.errors import NoDataError, ReportError

try:
    summarize(lines.iloc[0:0], target=380000.0)
except ReportError as exc:
    print(type(exc).__name__, "is a ReportError:", exc)
    print(isinstance(exc, NoDataError))

```

```

NoDataError is a ReportError: no sales lines for this month
True

```

Logging instead of print

`print` writes to the screen and disappears. **Logging** records messages with a **level** (how serious), a timestamp, and the name of the module that wrote them, and can send them to the screen, a file, or a monitoring system, all configured in one place:

Level	Use for	Example from the report
DEBUG	detail you only want while investigating	the SQL parameters used
INFO	normal progress worth a record	"read 35 sales lines for 2025-12"
WARNING	something odd that was handled	an API call failed and will be retried
ERROR	this run failed	"no sales lines for this month"
CRITICAL	the whole system is in trouble	rarely used in reports

The pattern has two halves. Each module asks for a named logger and writes messages. Only the program's entry point decides where messages go and which levels to show:

```

"""Command-line entry point: riverstone-report --month 2025-12"""
from __future__ import annotations

import argparse
import logging
import sys

from sqlalchemy import create_engine

from riverstone_report import db, excel, transform
from riverstone_report.config import ReportConfig
from riverstone_report.errors import ReportError

log = logging.getLogger("riverstone_report")

def run(config: ReportConfig) -> None:
    engine = create_engine(config.database_url)
    lines = db.fetch_sales_lines(engine, config.month, config.next_month)
    log.info("read %d sales lines for %s", len(lines), f"{config.month:%Y-%m}")
    summary = transform.summarize(lines, db.fetch_target(engine, config.month))
    path = excel.write_report(summary, transform.revenue_by_category(lines),
                             transform.top_customers(lines, config.top_n), config.output_path)
    log.info("net revenue %.2f from %d orders; wrote %s", summary.revenue, summary.orders, path)

def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(prog="riverstone-report", description="Build Riverstone's monthly sales report.")
    parser.add_argument("--month", required=True, help="the month to report, as YYYY-MM")
    parser.add_argument("--verbose", action="store_true", help="show debug messages")
    args = parser.parse_args(argv)
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                        format="%(asctime)s %(levelname)s %(name)s: %(message)s")
    try:
        run(ReportConfig.from_env(args.month))
    except ReportError as exc:
        log.error("%s", exc)
        return 1
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

- `logging.getLogger(__name__)` names each logger after its module (`riverstone_report.crm_client`), so a log line tells you where it came from.
- `log.info("read %d sales lines", len(lines))` passes values separately from the message. Logging only builds the text if the level is shown, and log tools can group identical messages.
- `logging.basicConfig(...)` is called once, in `main` . A module that calls it would override the settings of whatever program imports it.

The command now behaves like software a scheduler can trust:

```
# terminal, in any folder, with RIVERSTONE_DATABASE_URL set
$ uv run --project path/to/riverstone-report riverstone-report --month 2025-12
2026-09-17 03:30:34,350 INFO riverstone_report: read 35 sales lines for 2025-12
2026-09-17 03:30:34,370 INFO riverstone_report: net revenue 439823.50 from 18 orders; wrote
reports/riverstone_monthly_2025-12.xlsx
exit code: 0
$ uv run --project path/to/riverstone-report riverstone-report --month 2026-01
2026-09-17 03:30:35,024 INFO riverstone_report: read 0 sales lines for 2026-01
2026-09-17 03:30:35,025 ERROR riverstone_report: no sales lines for this month
exit code: 1
$ uv run --project path/to/riverstone-report riverstone-report --month Dec-2025
2026-09-17 03:30:35,588 ERROR riverstone_report: month must look like YYYY-MM, got 'Dec-2025'
exit code: 1
```

(The `exit code:` lines were printed by `echo "exit code: $?"` after each command.) January now fails with a reason and exit code 1, and writes nothing, so December's report is safe. Chapter 20's scheduled jobs use exactly this exit code to send "the report failed" alerts.

Watch out: never log secrets. `log.info("connecting to %s", config.database_url)` writes the database password into a log file that many more people can read than the password itself. Log *which* database (`db.example/riverstone`), not the full connection string, and never log API keys or personal data.

29.9 Robust API clients

Chapter 2 showed what an API is. Real APIs add three complications that a notebook call ignores, and every one of them appears in production:

1. **Pagination.** An API returns large lists in **pages** (10, 100, or 1,000 items at a time) and tells you how to ask for the next one.
2. **Transient failures.** Servers restart, networks drop, gateways time out. The same request a second later usually works. These are errors worth **retrying**.
3. **Rate limits.** APIs cap how many requests you may make. Exceed it and you get **HTTP 429 Too Many Requests**, often with a **Retry-After** header saying how many seconds to wait.

The practice API

Riverstone's CRM (Chapter 3) isn't connected to anything you can reach, so the companion folder has a local stand-in, `mock_crm_api.py`. It serves the 43 leads from `riverstone_2025` at `GET /leads?page=N&page_size=M`, requires the header `X-API-Key: demo-key`, and misbehaves on purpose: the first request for page 2 gets **503 Service Unavailable**, and the first request for page 3 gets **429** with `Retry-After: 1`. It runs on your own machine and sends nothing anywhere.

A response looks like this:

```
{"page": 5, "items": [{"lead_id": 41, "created_at": "2025-01-06T09:35:00", "company_name": "Crown
Hotels", "source": "Referral"}], "next_page": null, "total": 43}
```

(One item shown; page 5 has three.)

The naive client

Here's how most first API calls are written:

```
import requests
from mock_crm_api import start_server

server, base_url = start_server(8029)      # starts the mock API on port 8029 of your own machine

leads, page = [], 1
while page is not None:
    response = requests.get(f"{base_url}/leads", params={"page": page, "page_size": 10},
                            headers={"X-API-Key": "demo-key"})
    data = response.json()
    leads.extend(data["items"])
    page = data["next_page"]
print(len(leads))
```

```
KeyError: 'items'
```

`KeyError: 'items'`. Page 2 returned a 503 whose body is `{"error": "temporarily unavailable"}`, the code never checked the status, and it crashed looking for a key that isn't there. Worse variants don't crash: they skip the page and report 33 leads as if that were all of them. And this call has **no timeout**: `requests` waits forever by default, so if the server stops responding, a scheduled job hangs until someone notices.

The robust client

```
"""A careful client for the CRM's leads API: pagination, timeouts, retries with backoff, and rate
limits."""
from __future__ import annotations

import logging
import random
import time
from collections.abc import Callable, Iterator
from typing import Any

import requests

from riverstone_report.errors import CrmApiError

log = logging.getLogger(__name__)

RETRY_STATUSES = {429, 500, 502, 503, 504}

class CrmClient:
    def __init__(
        self,
        base_url: str,
        api_key: str,
        *,
        session: requests.Session | None = None,
        max_attempts: int = 4,
        backoff_seconds: float = 0.5,
        timeout: tuple[float, float] = (3.05, 10.0),
        sleep: Callable[[float], None] = time.sleep,
        rng: random.Random | None = None,
    ) -> None:
        self.base_url = base_url.rstrip("/")
        self.session = session or requests.Session()
        self.session.headers["X-API-Key"] = api_key
        self.max_attempts = max_attempts
        self.backoff_seconds = backoff_seconds
        self.timeout = timeout
        self.sleep = sleep
        self.rng = rng or random.Random()

    def _wait_time(self, attempt: int, response: requests.Response | None) -> float:
        if response is not None and response.status_code == 429:
            retry_after = response.headers.get("Retry-After", "")
            if retry_after.isdigit():
                return float(retry_after)
            base: float = self.backoff_seconds * 2 ** (attempt - 1)
            return round(base + self.rng.uniform(0, base / 2), 3)

    def get_json(self, path: str, params: dict[str, Any]) -> dict[str, Any]:
        url = f"{self.base_url}{path}"
        for attempt in range(1, self.max_attempts + 1):
            response: requests.Response | None = None
            try:
                response = self.session.get(url, params=params, timeout=self.timeout)
            except (requests.ConnectionError, requests.Timeout) as exc:
                problem = type(exc).__name__
            else:
                if response.status_code == 200:
                    data: dict[str, Any] = response.json()
                    return data
                if response.status_code not in RETRY_STATUSES:
                    raise CrmApiError(f"{url} returned {response.status_code}")
```

```

{response.text[:100]}")
    problem = f"HTTP {response.status_code}"
    if attempt == self.max_attempts:
        raise CrmApiError(f"{url} failed after {attempt} attempts (last: {problem})")
    wait = self._wait_time(attempt, response)
    log.warning("attempt %d for %s failed (%s); retrying in %.3f s", attempt, params,
problem, wait)
    self.sleep(wait)
    raise AssertionError("unreachable")

def iter_leads(self, page_size: int = 10) -> Iterator[dict[str, Any]]:
    page = 1
    while True:
        data = self.get_json("/leads", {"page": page, "page_size": page_size})
        yield from data["items"]
        if data["next_page"] is None:
            return
        page = data["next_page"]

```

How it works:

- **One requests.Session** reuses the network connection across pages and holds the API key header, instead of repeating it on every call.
- **timeout=(3.05, 10.0)** : give up if connecting takes more than about 3 seconds, or if the server goes silent for 10. Every network call in production code needs a timeout.
- **Which failures to retry.** 429 and the 5xx server errors are **transient**: the same request may work later. So are connection errors and timeouts. A **401** (bad key), 403 (not allowed), or 404 (wrong address) will fail identically every time, so the client raises `CrmApiError` immediately.
- **Exponential backoff**: wait 0.5 s after the first failure, then 1 s, then 2 s, doubling each time, so a struggling server gets breathing room. **Jitter** adds a random extra of up to half the wait, so a hundred clients that failed together don't all retry at the same instant.
- **Retry-After** on a 429 overrides the backoff: the server said how long to wait, so wait exactly that.
- **Give up eventually**: after `max_attempts` (4), raise an error that says what happened last. Retrying forever turns a failure into a hang.
- **iter_leads is a generator** (`yield from`): it hands back leads page by page as they arrive, and stops when `next_page` is `null`. A caller can write `for lead in client.iter_leads():` without knowing pages exist.
- **sleep and rng are parameters.** Normally they're `time.sleep` and a real random generator. Tests pass `sleeps.append` and a seeded generator, so a test of four retries runs instantly and checks the exact waits (`tests/test_crm_client.py`).

The same fetch, with the robust client, logging to the screen:

```
import logging, random, sys
from riverstone_report.crm_client import CrmClient

logging.basicConfig(stream=sys.stdout, level=logging.INFO,
                    format="%(levelname)s %(name)s: %(message)s", force=True)

server.shutdown()
server, base_url = start_server(8029)      # a fresh server, with its planned failures reset

client = CrmClient(base_url, "demo-key", rng=random.Random(29))
leads = list(client.iter_leads(page_size=10))
print(len(leads), "leads; last:", leads[-1]["company_name"], leads[-1]["created_at"])
```

```
WARNING riverstone_report.crm_client: attempt 1 for {'page': 2, 'page_size': 10} failed (HTTP 503);
retrying in 0.637 s
WARNING riverstone_report.crm_client: attempt 1 for {'page': 3, 'page_size': 10} failed (HTTP 429);
retrying in 1.000 s
43 leads; last: Daily Fresh Mart 2025-11-25T10:50:00
```

Both failures were logged, waited out, and retried, and all 43 leads arrived. ✓ The 43 matches Chapter 13's count of lead records (30 real leads plus 13 duplicates, which is Chapter 13's Pattern 3 problem, not the client's). The waits were 0.637 s (0.5 s backoff plus jitter) and exactly 1 s for the 429.

Fetching 43 leads from the mock CRM API: two failures, no lost data



Real run on the local mock server; the wait times are from the chapter's output.

Retry: 429, 500, 502, 503, 504,
connection errors, timeouts

Don't retry: 400, 401, 403, 404
(retrying can't fix them)

Wait: $0.5 \text{ s} \times 2^{(\text{attempt}-1)}$,
plus random jitter up to half

429 with Retry-After: wait that long

Give up after 4 attempts: raise CrmApiError

Always set a timeout: (3.05, 10) s

Figure 29.4 – Retry what might work next time, wait sensibly, and stop when the API says there are no more pages.

And the failure the client must not retry:

```
bad_client = CrmClient(base_url, "wrong-key", rng=random.Random(29))
try:
    list(bad_client.iter_leads())
except ReportError as exc:
    print(type(exc).__name__ + ":", exc)
server.shutdown()
```

```
CrmApiError: http://127.0.0.1:8029/leads returned 401: {"error": "invalid or missing API key"}
```

No retries, no waiting: a wrong key won't become right in two seconds.

Watch out: retries must be safe to repeat. Reading leads twice does no harm. Creating an order twice does. A request that can be repeated without changing the outcome is **idempotent**. GET requests normally are; POST requests that create things normally aren't. Before retrying a write, check whether the API supports an **idempotency key** (a unique id sent with the request so the server ignores duplicates). Chapter 51 covers writing to other systems safely.

Tool note. `urllib3`'s `Retry` class can add retries and backoff to a `requests.Session` in a few lines, and the `tenacity` package provides retry decorators. `httpx` is a newer HTTP client with a similar interface that also supports asynchronous code. Writing the loop by hand once, as here, is how you learn what those tools configure.

29.10 Code review

Code review means another person reads a change before it's merged into the shared code. It catches bugs, but its bigger value is that at least two people understand every part of the system, and that standards spread without anyone writing a manual. On most teams, changes arrive as **pull requests** (Chapter 26): a set of commits with a description, which reviewers comment on line by line.

As the author

1. **Keep changes small:** one purpose per pull request. A reviewer can check 200 lines properly; 2,000 lines get a skim and an approval.
2. **Explain the why** in the description, and how you tested it: *"Counts orders with `nunique()`, not rows. Added `test_summarize_counts_orders_not_lines`. Reconciles to ₹439,823.50 for December."*
3. **Run the checks first:** tests, type checker, formatter. Don't spend a reviewer's time on what a tool can find.
4. **Point to the risky part:** *"Please look closely at the retry logic."*

As the reviewer

Ask	For example, in this chapter's package
Is it correct for the business rule?	Orders counted with <code>nunique</code> ; cancelled orders excluded (by the <code>sales_lines</code> view)
What happens on the failure path?	Empty month raises <code>NoDataError</code> ; 401 not retried
Could it leak a secret or personal data?	No password in code or logs; <code>.env</code> in <code>.gitignore</code>
Is it tested, and do the tests test the rule?	A test named after the rule, with hand-checkable data
Will the next person understand it?	Names say what things are; comments say why
Does it handle scale?	Query filters by month in SQL, not in pandas
Is it the simplest thing that works?	A dataclass, not a class hierarchy

How to comment. Comment on the code, never the person. Ask rather than instruct when you might be missing context ("What happens if the target row is missing here?"), label optional suggestions ("nit:" for trivial style points), and say what's good as well. Approve when the change is correct and safe, not when it's exactly how you'd have written it.

Real-life example: the review comment that saved a quarter's numbers. A common catch in analytics teams: a pull request changes a revenue calculation to "include pending orders so the dashboard updates sooner". The code is clean and the tests pass, because the tests were updated too. The reviewer asks one question: "Has finance agreed to count pending orders as revenue?" They hadn't. Review isn't only about code; it's the moment someone checks that a business rule changed on purpose.

Formatting and linting

Arguments about spacing and quote styles waste review time, so teams hand them to tools. A **formatter** rewrites code into a consistent style automatically, and a **linter** flags likely mistakes (unused imports, undefined names, a bare `except`). **Ruff** does both and is widely used with `uv` (`uv add --dev ruff`, then `uv run ruff check` and `uv run ruff format`). Run them before review, and in automated checks on every pull request (Chapter 32 does the same for SQL).

Common mistakes and how to spot them

Mistake	Symptom	Fix
All code at the top level of one file	Importing it to test anything connects to the database and writes files	Functions in modules; a <code>__main__</code> block or entry point only in the CLI
Password or API key in the code	It appears in Git history, screenshots, and emails	Environment variables or a secrets manager; <code>.env</code> in <code>.gitignore</code>
Month or file name typed into the code	Every run edits code; reports overwrite each other	Command-line arguments; file names built from the settings
Installing packages into the system Python	Projects break each other; "works on my machine"	One virtual environment per project
No lockfile, or not committing it	A new install gets different versions and behaves differently	Commit <code>uv.lock</code> (or pinned requirements); install with <code>uv sync --locked</code>
SQL built by adding strings	Breaks on quotes; SQL injection	Query parameters
Bare <code>except:</code> or <code>except Exception: pass</code>	Wrong results with no error; Ctrl+C doesn't stop the program	Catch named exceptions you can handle; let bugs crash
Turning a failure into a default value (<code>target = 0</code>)	Plausible-looking wrong numbers	Raise, or use <code>None</code> and handle it visibly
<code>print</code> in scheduled code	No timestamps, no levels, nothing saved; scheduler thinks it succeeded	<code>logging</code> , and a non-zero exit code on failure

Mistake	Symptom	Fix
Calling <code>logging.basicConfig</code> inside a library module	Other programs' logging settings get overridden	Configure logging once, in the entry point
Tests that need the database for every calculation	Tests are slow, flaky, and skipped	Pure functions tested with tiny DataFrames; one integration test
Test data too large to check by hand	Nobody can tell whether the expected value is right	Five rows you can add up in your head
Tests that only cover the happy path	Failures surprise you in production	<code>pytest.raises</code> for bad input, empty data, and missing settings
No timeout on an HTTP call	A job hangs forever when the server stops responding	Always pass <code>timeout=</code>
Not checking the HTTP status	<code>KeyError</code> , or silently missing pages	Check <code>status_code</code> ; raise or retry
Retrying every error, forever, immediately	Bans for hammering the API; hangs; duplicate writes	Retry only transient errors, with backoff, a limit, and <code>Retry-After</code>
Retrying non-idempotent requests	Duplicate records in the other system	Retry only safe requests, or use idempotency keys
Classes for everything	Code is harder to follow than the script it replaced	Functions for calculations; data classes for bundles; classes for shared state
Ignoring type checker errors about <code>None</code>	Crash on the first month with no target	Handle the <code>None</code> case explicitly

In the real world: the report that said January was zero

It's Tuesday, 3 February 2026. At 9:10 the managing director forwards Riverstone's monthly report to Anita Rao with one line: "Revenue zero in January?"

Meera Iyer runs the report. Since Imran left, it's a copy of his script, scheduled with Windows Task Scheduler to run on the second of each month. She opens the log folder: there isn't one. The scheduler's history says the task **completed successfully**. The shared drive has one file, `Monthly_Report_Dec_FINAL.xlsx`, dated yesterday, showing January with ₹0 revenue and a blank percentage of target.

It takes her an hour to piece together:

1. The load from the ERP to the reporting database had been failing since the last day of December, and nobody was alerted. The reporting copy had no January orders at all.
2. The script found no rows, summed them to zero, and divided zero by a target of zero, because `except: target = 0` had also hidden the fact that January's target row hadn't been entered yet.
3. It printed `done` and exited with code 0, so the scheduler recorded success.
4. It saved over the file with the fixed name, so December's good report was gone too. (IT restored it from the previous night's backup, as Chapter 2 hoped someone would.)

The data team reloads the database, and Meera reruns the script by hand. January's report goes out at 2 p.m., with the ₹104,210 of January invoices that Chapter 3's report shows.

Then she fixes the cause, not the symptom, in three pull requests over the following two weeks, each reviewed by the data engineer:

- **Fail loudly.** The report became the package in this chapter. An empty month raises `NoDataError`, a missing target shows "no target set" instead of 0, and every failure exits with code 1. The scheduler now emails her when the exit code isn't 0.
- **Stop guessing.** Tests for the rules that bit her: orders counted, not lines; empty month; missing target; December rolling into January. And one reconciliation test against a month everyone agrees on.
- **Make it runnable by anyone.** The password moved into an environment variable on the report server. `uv.lock` pins the versions, so the server and her laptop run the same code. Output files are named by month, so no report can overwrite another.

On 2 March, the nightly load fails again. At 07:00 the report runs, logs `ERROR riverstone_report: no sales lines for this month`, exits with code 1, and Meera has an email before she reaches her desk. The MD never sees a wrong number.

What she tells Anita: *"The report used to succeed even when it had nothing to report. Now, if the data isn't there, it stops, tells me, and leaves last month's file alone. It also has tests for the rules finance cares about, so when someone changes a calculation, we'll know the same day."*

Notice that the fix wasn't clever code. It was ordinary habits: explicit failures, a meaningful exit code, tests for real rules, settings outside the code, and a second pair of eyes.

Tools

Versions used for this chapter, checked in September 2026:

- **Python 3.12.3.** Everything here works on 3.12 and later.
 - **uv 0.12.15**, installed from `docs.astral.sh/uv`. `uv` releases frequently; check the documentation for current installation commands.
 - **Packages** (as locked in `uv.lock`): `pandas 3.0.5`, `openpyxl 3.1.5`, `SQLAlchemy 2.0.54`, `psycopg 3.3.5`, `requests 2.34.2`; for development, `pytest 9.1.1`, `mypy 2.3.1`, `pandas-stubs`, `types-requests`, `types-openpyxl`.
 - **An editor with Python support**, such as VS Code with the Python extension (runs tests, shows type errors as you type).
 - **Ruff** (optional) for formatting and linting.
 - **Git** (Chapter 26) for version control and pull requests.
 - **Companion files** in `ch29/`: `start/monthly_report.py` (the starting script), `riverstone-report/` (the finished package with `pyproject.toml`, `uv.lock`, source, and 23 tests), `mock_crm_api.py` and `leads.json` (the practice API), and `ch29_check.py` (checks the chapter's numbers).
-

The project: turn a script into a tested package

Goal: a script you (or your team) depend on becomes a package that someone else can install and run with one command, with tests that protect its business rules.

Option A: your own script. Choose a real script or notebook that produces something people use. Work in a private copy; remove passwords, server names, and personal data before sharing it with anyone, including in a portfolio.

Option B: Riverstone. Start from `start/monthly_report.py` and rebuild the package yourself without looking at `riverstone-report/`, then compare.

Steps:

1. **List the problems** in the original, using section 29.1's table as a checklist.
2. **Create the project** with `uv init --package`, and add dependencies with `uv add`. Commit `pyproject.toml` and `uv.lock`.
3. **Split the jobs** into modules: configuration, reading, transforming (pure), writing, and a command-line entry point.
4. **Move settings out of the code:** arguments for each run, environment variables for secrets and machine-specific settings.
5. **Write tests first for the rules that matter**, with a fixture of five to ten rows you can check by hand. Include at least three failure cases.
6. **Add type hints** and get `uv run mypy` passing in strict mode.
7. **Replace print with logging**, catch your own exceptions at the entry point, and return an exit code.
8. **Reconcile** one real run against a number you already trust.
9. **Write a README:** what it does, how to install (`uv sync`), how to run, which environment variables it needs, how to run the tests.
10. **Ask someone to review it** with section 29.10's checklist, and act on the comments.

Deliverables: the repository, the test output, the reconciliation, and a half-page note listing each problem from step 1 and how it was fixed.

Stretch goals:

- Add a `--dry-run` flag that runs everything except writing the file.
- Pull the month's new leads from the mock CRM API with `CrmClient` and add a "Leads by source" sheet.
- Add Ruff and a pre-commit hook that runs Ruff and the tests before every commit.
- Run the tests automatically on every push with a CI service (Chapter 26 introduces them).

You've got it when...

- ☐ I can say when a script should become a package, and when it shouldn't.

- ☐ My business calculations are pure functions that I can run on a five-row DataFrame.
 - ☐ My projects have a `src/` layout, a `pyproject.toml`, and a committed lockfile.
 - ☐ I can recreate a project's exact environment on a new machine with one command.
 - ☐ No password or key appears in my code, my Git history, or my logs.
 - ☐ I use data classes for bundles of settings and results, and classes only when data and behavior belong together.
 - ☐ I add type hints and fix what `mypy` finds, especially `None` cases.
 - ☐ I write tests for business rules, edge cases, and failures, using fixtures and `parametrize`.
 - ☐ My scheduled code logs with levels and exits with a non-zero code on failure.
 - ☐ My API clients set timeouts, follow pages, retry only transient errors with backoff, and respect `Retry-After`.
 - ☐ I review code with a checklist, and write pull requests that are quick to review.
-

Recap

- **Software** is code others can install, run, understand, change, and trust. The time to write it is when people depend on the output, it runs on a schedule, or someone else will change it.
 - Split code into **functions** and **modules** with one job each. Keep business rules in **pure functions**; put databases, files, and networks at the edges.
 - Use the `src/` layout, a `pyproject.toml`, and **environment variables** for secrets and machine settings. Use query **parameters**, never string-built SQL.
 - A **virtual environment** isolates a project's packages; a **lockfile** pins every version. `uv` manages both (`uv add`, `uv sync --locked`, `uv run`); `venv` with pinned requirements is the universal fallback.
 - **Data classes** bundle values with names and types; **properties** calculate instead of storing; `frozen=True` prevents accidental changes.
 - **Type hints** document intent, and `mypy` checks them without running code, catching bugs such as unhandled `None`.
 - `pytest` runs `test_*` functions; **fixtures** supply fresh test data; **parametrize** tests many inputs; `pytest.raises` tests failures. Many fast unit tests, one reconciliation against a known total.
 - Catch **named exceptions**, never bare `except`. Define your own (`ReportError`), handle them at the entry point, and return an **exit code**. Use **logging** with levels, configured once, and never log secrets.
 - **Robust API clients** use a session, **timeouts**, **pagination**, and **retries** for transient errors (429, 5xx, connection errors) with **exponential backoff**, **jitter**, `Retry-After`, and a limit. Retry only **idempotent** requests.
 - **Code review**: small, explained changes; reviewers check rules, failures, secrets, tests, and clarity. Formatters and linters handle style.
-

Practice exercises

Run code exercises from the companion folder `ch29/`, with `riverstone-report/src` on the path (as in section 29.2) or inside the package with `uv run`. Exercises that need the database say so.

Warm-up

1. Write a pure function `net_revenue(quantity: int, unit_price: float, discount_pct: float) -> float` that returns the net value of one order line, rounded to 2 decimal places. Check it on order 10005's line from Chapter 28: 15 boxes at ₹430 with 5% discount.
2. Predict what each call prints or raises, then run them: `parse_month("2025-02")`, `parse_month("2025-2")`, `parse_month("2025/02")`.
3. Read the starting script's `except:` block. Name three different problems that would all end up as `target = 0`.

Core

4. Add a function `average_order_value(lines: pd.DataFrame) -> float` to your own copy of `transform.py`: net revenue divided by the number of orders, rounded to 2 decimals. What does it return for the five-line test DataFrame? Write its test.
5. Write a parametrized test for `net_revenue` from exercise 1 with three cases, including a 0% and a 100% discount.
6. Change the logging setup so that `INFO` and above go to the screen and `DEBUG` and above also go to a file called `report.log`. Show the code.
7. Use `CrmClient` against the mock API to count leads by `source`, most common first. How many leads came from the website?
8. Extend `ReportConfig.from_env` to read an optional `RIVERSTONE_TOP_N`. It must be a whole number from 1 to 20; anything else raises `ConfigError`. Show what happens with `"25"` and `"five"`.
9. mypy reports: `error: Item "None" of "float | None" has no attribute "__round__" for round(summary.pct_of_target)`. Explain the problem in one sentence and fix the line.

Stretch

10. A server sends `Retry-After: Wed, 21 Oct 2026 07:28:00 GMT` (a date instead of a number of seconds). What does `CrmClient._wait_time` do with it? Write a test that proves your answer.
11. Make `write_report` safe against half-written files: write to a temporary file in the same folder, then rename it to the final name. Explain why that matters for a report someone might open while it's being written.
12. The mock API can be started with a different failure plan: `start_server(failures={2: [500, 500, 500, 500]})`. Predict what `CrmClient` with the default settings does, then run it. How many seconds does it spend waiting, roughly?

Think about it (no code needed)

- l3. A colleague says: "Logging is overkill; my script prints what it's doing and I can see it." When is she right, and when isn't she?
- l4. Give two situations where turning a script into a package with tests would be a waste of time.
- l5. You're reviewing a pull request that changes `revenue_by_category` to exclude the Furniture category "because it's being discontinued", with the test updated to match. What do you ask before approving?

Key terms

software · script · pure function · module · package · `__name__` · SQL injection · query parameter · `src/` layout · `pyproject.toml` · entry point · environment variable · `.env` file · secrets manager · virtual environment · `venv` · lockfile · pinned requirements · `uv` · `uv.lock` · dependency group · class · data class · immutable · property · alternative constructor · type hint · type checker · `mypy` · test · `pytest` · assertion · fixture · parametrize · unit test · integration test · coverage · exception · custom exception · exit code · logging · log level · logger · pagination · transient failure · rate limit · HTTP 429 · `Retry-After` · timeout · retry · exponential backoff · jitter · session · generator · idempotent · idempotency key · code review · pull request · formatter · linter · Ruff

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 30, Inference & Experiments**, uses Python for statistics; write its analyses as tested functions from the start.
 - **Chapter 20, Automating Reports & Delivering Insights**, schedules this package and emails people when its exit code says it failed.
 - **Chapter 33, The Computer Science You Actually Need**, explains why some Python code is slow and how to measure and fix it.
 - **Chapter 32, Analytics Engineering with dbt**, applies these habits to SQL: version control, tests, review, and automated checks.
 - **Chapter 46, Pipelines & Orchestration**, and **Chapter 56, MLOps**, build on packages, lockfiles, logging, and retries at production scale.
 - **Chapter 72, Python & pandas Question Bank**, has interview questions on project structure, testing, error handling, and API clients.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1.

```
import sys
sys.path[:0] = ["riverstone-report/src", "."]

def net_revenue(quantity: int, unit_price: float, discount_pct: float) -> float:
    """Net value of one order line, rounded to paise."""
    return round(quantity * unit_price * (1 - discount_pct / 100), 2)

print(net_revenue(15, 430, 5))
```

6127.5

₹6,127.50, the same as the hand check in section 28.7. ✓ It's pure: no database, no global variables, and the same inputs always give the same output.

2.

```
from riverstone_report.config import parse_month
from riverstone_report.errors import ConfigError

for text in ["2025-02", "2025-2", "2025/02"]:
    try:
        print(text, "->", parse_month(text))
    except ConfigError as exc:
        print(text, "-> ConfigError:", exc)
```

```
2025-02 -> 2025-02-01
2025-2 -> 2025-02-01
2025/02 -> ConfigError: month must look like YYYY-MM, got '2025/02'
```

"2025-2" works, because `int("2")` is 2. "2025/02" fails because splitting on `-` gives one part, which can't be unpacked into `year, month`, a `ValueError` that `parse_month` turns into a `ConfigError`. If you want to insist on two-digit months, that's a rule to add *and* a test to write.

3. Any three of: the `sales_targets` table doesn't exist in the database the script connected to; the target row for that month hasn't been entered yet (`iloc[0, 0]` raises `IndexError`); a typo in the SQL; the database connection dropped; the value can't be converted to float; the user pressed Ctrl+C at that moment (a bare `except` even catches `KeyboardInterrupt`). They all need different responses, and the script gave every one of them the same wrong answer.

4.

```
import pandas as pd

def average_order_value(lines: pd.DataFrame) -> float:
    orders = lines["order_id"].nunique()
    if orders == 0:
        raise ValueError("no orders to average")
    return round(float(lines["net_revenue"].sum()) / orders, 2)

lines = pd.DataFrame({
    "order_id":      [1, 1, 2, 3, 3],
    "customer_name": ["Sharma Hardware", "Sharma Hardware", "Metro Mart", "Green Leaf Hotels",
"Green Leaf Hotels"],
    "category":      ["Storage", "Kitchen", "Storage", "Kitchen", "Furniture"],
    "net_revenue":    [10000.0, 2500.0, 6000.0, 1500.0, 5000.0],
})
print(average_order_value(lines))
```

8333.33

₹25,000 ÷ 3 orders = ₹8,333.33. The common wrong answer is `lines["net_revenue"].mean()`, which averages *lines* (₹5,000). The test, in `tests/test_transform.py`:

```
def test_average_order_value_divides_by_orders(lines):
    assert average_order_value(lines) == 8333.33

def test_average_order_value_of_no_orders(lines):
    with pytest.raises(ValueError):
        average_order_value(lines.iloc[0:0])
```

5.

```
import pytest

from riverstone_report.transform import net_revenue

@pytest.mark.parametrize("quantity, unit_price, discount_pct, expected", [
    (15, 430.0, 5.0, 6127.50),
    (20, 450.0, 0.0, 9000.00),
    (10, 380.0, 100.0, 0.00),
])
def test_net_revenue(quantity, unit_price, discount_pct, expected):
    assert net_revenue(quantity, unit_price, discount_pct) == expected
```

Three results in pytest's output, one per row. (This assumes you added `net_revenue` to `transform.py`.) Consider adding a case that should *fail*, such as a discount of 120%, and deciding whether `net_revenue` should raise for it.

6. Configure a handler for each destination, each with its own level, and set the logger's level to the lowest:

```

import logging

formatter = logging.Formatter("%(asctime)s %(levelname)s %(name)s: %(message)s")

console = logging.StreamHandler()
console.setLevel(logging.INFO)
console.setFormatter(formatter)

logfile = logging.FileHandler("report.log", encoding="utf-8")
logfile.setLevel(logging.DEBUG)
logfile.setFormatter(formatter)

root = logging.getLogger()
root.setLevel(logging.DEBUG)
root.addHandler(console)
root.addHandler(logfile)

```

This replaces `logging.basicConfig(...)` in `main`. A common mistake is setting only the handlers' levels: the logger's own level (default `WARNING`) is checked first, so `DEBUG` messages would never reach the file.

7.

```

import random
from collections import Counter

from mock_crm_api import start_server
from riverstone_report.crm_client import CrmClient

server, base_url = start_server(8029, failures={}) # no planned failures for this exercise
client = CrmClient(base_url, "demo-key", rng=random.Random(7))
by_source = Counter(lead["source"] for lead in client.iter_leads(page_size=20))
server.shutdown()
for source, count in by_source.most_common():
    print(f"{source:<18} {count}")
print("total", sum(by_source.values()))

```

```

Website           27
Referral          5
Trade fair        5
Cold call         3
IndiaMART listing 3
total 43

```

27 of the 43 lead records came from the website. The counts include Chapter 13's duplicate lead records, because the API returns what the CRM holds; deduplicate (Chapter 13, Pattern 3) before reporting leads by source to anyone.

8.

```

top_n_text = env.get("RIVERSTONE_TOP_N", "5")
if not top_n_text.isdigit() or not 1 <= int(top_n_text) <= 20:
    raise ConfigError(f"RIVERSTONE_TOP_N must be a whole number from 1 to 20, got {top_n_text!r}")
# then pass top_n=int(top_n_text) to cls(...)

```

With this added to `from_env`, a value of `"25"` raises `ConfigError: RIVERSTONE_TOP_N must be a whole number from 1 to 20, got '25'`, and `"five"` raises the same message with `'five'`. `isdigit()` is checked first, so `int()` is never called on text that isn't a number.

9. `pct_of_target` is `None` when a month has no target, and `round(None)` fails, so the code must decide what to show in that case: `round(summary.pct_of_target)` if `summary.pct_of_target` is not `None` else `"no target"`. After the `is not None` check, `mypy` knows the value is a float.

10. `"Wed, 21 Oct 2026 07:28:00 GMT".isdigit()` is `False`, so `_wait_time` ignores the header and falls back to exponential backoff. The HTTP standard allows `Retry-After` to be either a number of seconds or a date. The test, added to `tests/test_crm_client.py`:

```
def test_retry_after_as_a_date_falls_back_to_backoff():
    sleeps = []
    client = make_client([FakeResponse(429, headers={"Retry-After": "Wed, 21 Oct 2026 07:28:00 GMT"}),
                          FakeResponse(200, {"items": [], "next_page": None})], sleeps)
    list(client.iter_leads())
    assert len(sleeps) == 1 and 0.5 <= sleeps[0] <= 0.75
```

With `backoff_seconds=0.5`, the first wait is 0.5 s plus jitter of up to 0.25 s. Supporting the date form is a reasonable improvement: parse it with `email.utils.parsedate_to_datetime` and wait until that moment.

11.

```
import os
import tempfile

# inside write_report, replacing the direct write:
fd, temp_name = tempfile.mkstemp(suffix=".xlsx", dir=path.parent)
os.close(fd)
temp_path = Path(temp_name)
try:
    with pd.ExcelWriter(temp_path, engine="openpyxl") as writer:
        ...
        os.replace(temp_path, path)
except BaseException:
    temp_path.unlink(missing_ok=True)
    raise
```

If the program crashes or is stopped halfway, a direct write leaves a damaged workbook under the real name, and anyone opening it (or an email job attaching it) gets a broken file. Writing to a temporary file and then replacing the final file means the real name only ever points to a complete report.

`os.replace` works as a single step when both files are in the same folder, which is why the temporary file is created in `path.parent`.

12. Four 500s for page 2 exhaust all four attempts, so the client raises `CrmApiError` after waiting three times: about 0.5, 1, and 2 seconds plus jitter, between 3.5 and 5.25 seconds in total. Page 1's 20 leads are fetched first, but because `iter_leads` is consumed by `list(...)`, the caller gets the exception, not a partial list:

```

import logging, time
from riverstone_report.errors import ReportError

logging.basicConfig(stream=sys.stdout, level=logging.WARNING,
                    format="%(levelname)s %(name)s: %(message)s", force=True)

server, base_url = start_server(8029, failures={2: [500, 500, 500, 500]})
client = CrmClient(base_url, "demo-key", rng=random.Random(12))
started = time.perf_counter()
try:
    leads = list(client.iter_leads(page_size=20))
except ReportError as exc:
    print(type(exc).__name__ + ":", exc)
waited = time.perf_counter() - started
print("between 3.5 and 5.3 seconds:", 3.5 <= waited <= 5.3)
server.shutdown()

```

```

WARNING riverstone_report.crm_client: attempt 1 for {'page': 2, 'page_size': 20} failed (HTTP 500);
retrying in 0.619 s
WARNING riverstone_report.crm_client: attempt 2 for {'page': 2, 'page_size': 20} failed (HTTP 500);
retrying in 1.329 s
WARNING riverstone_report.crm_client: attempt 3 for {'page': 2, 'page_size': 20} failed (HTTP 500);
retrying in 2.666 s
CrmApiError: http://127.0.0.1:8029/leads failed after 4 attempts (last: HTTP 500)
between 3.5 and 5.3 seconds: True

```

The three warnings show the growing waits; the fourth failure ends the attempts. `CrmApiError` is a `ReportError`, so catching the base class works.

13. She's right for code she runs by hand and watches, once. She isn't when it runs on a schedule or on a server: nobody is watching the screen, prints have no timestamps or levels, nothing is kept after the window closes, and a scheduler can't tell a printed error from success. Logging costs two lines per module, and turns the same messages into a record you can search on the day something goes wrong.

14. Two of: a one-off analysis for a single question that won't be repeated; exploring a new dataset in a notebook to decide whether it's worth anything; a throwaway conversion of a file that will be deleted afterwards; a quick prototype whose purpose is to find out whether an idea works at all. The test is whether anyone will run, depend on, or change it again. When the answer changes to yes, that's the moment to refactor.

15. Whether the business decision has been made and by whom (Anita? finance?); whether "discontinued" means past Furniture revenue should disappear from historical reports too, or only future sales stop; whether this should be a filter in the report at all, or a product status in the data so every report agrees; and whether the total revenue on the Summary sheet will still reconcile with the categories. Updating a test to match changed code proves only that the code does what it now says, not that it should.